DEFINITIVE GUIDE · SUPERSEDES ALL EARLIER FLASHING NOTES

Flashing the Eagle 101

Tanna TechBiz Eagle 101 + NVIDIA Jetson Nano production module (P3448-0002)
Unit serial 1010425035310 · EchoPath deployment

BOARD CONFIG	L4T VERSION	HOST OS	RECOVERY PINS
`jetson-nano-devkit-emmc`	`R32.7.5`	`Ubuntu 18.04`	`3 (FC_REC) - 4 (GND)`

Every unknown is now resolved
The board-configuration name, the recovery-pin numbers and the exact file list all come from Tanna's official flashing guide. Nothing in this document is guessed.

HOW TO READ THE SOURCE LABELS

TANNA from the vendor's flashing guide · NVIDIA standard L4T procedure · ECHOPATH specific to this project

The single most useful surprise TANNA

There is no proprietary Tanna BSP to obtain. All four required files come from NVIDIA's public Jetson Linux Archive. Tanna's contribution is documenting *which* stock pieces to combine, plus one device-tree file. You do not need to wait for a reply from them to begin.

Part 0 · Do you even need to flash?

Thirty minutes, and it may make the rest of this document unnecessary.

0.1 Power on with a monitor ECHOPATH

Connect HDMI first, then a USB keyboard, then the 5 V / 4 A barrel supply last. Wait up to two minutes — first boot is slow.

What you see	Meaning
Nothing, fan not spinning	Check the supply and that the module is fully seated, then treat as blank → flash
Fan spins, screen black	No operating system → **flash**
NVIDIA logo → text → login screen	An OS is present → continue to 0.2

0.2 The camera test — this is what decides it

Power off, connect the Arducam to CSI port 0, power on. Then:

```
ls /dev/video0
cat /etc/nv_tegra_release        # which L4T version is installed?
df -h /                          # how much eMMC is free?
```

Result	Action
`/dev/video0` exists	**Do not flash.** Skip to Part 8 (SDK) and Part 9 (SD card)
`No such file or directory`	Wrong device tree → **flash**

A board can boot perfectly on generic NVIDIA firmware and still have no camera, because this carrier routes the CSI lanes differently. Booting is not the test; the camera is.

Part 1 · What you need

HARDWARE

- ☐ Eagle 101 with the Jetson Nano module seated, and the fan connected
- ☐ **5 V / 4 A barrel supply** — an under-powered adapter causes random reboots that look like software faults
- ☐ **USB Type-A to Type-B cable**, data-capable. A charge-only cable is the single commonest failure
- ☐ A jumper (or a paperclip) for the recovery pins
- ☐ HDMI cable, monitor, USB keyboard — for first boot
- ☐ Your 128 GB SanDisk microSD

HOST COMPUTER TANNA

- ☐ **Ubuntu 18.04, x86_64, physical machine**
- ☐ ~40 GB free disk space
- ☐ An NVIDIA Developer account (free) to download L4T

> **Tanna explicitly advises against these** TANNA
>
> *"We do not recommend using virtual machines and ARM architecture Ubuntu."*
>
> That rules out VirtualBox, VMware, **and WSL2** (which is VM-backed), and any Apple Silicon Mac. During flashing the board disconnects and reappears on the USB bus several times; virtualised USB passthrough commonly drops it mid-transfer.
>
> **Note also: Ubuntu 18.04 specifically**, not 20.04. Since 18.04 is no longer in the Microsoft Store, the WSL2 route is doubly impractical. Use a lab machine, or dual-boot 18.04 on your own PC.

Part 2 · Download the four files TANNA

All from **NVIDIA's Jetson Linux Archive** — `developer.nvidia.com/embedded/jetson-linux-archive` — under **L4T R32.7.5**. Log in with a free developer account.

File	What it is
`Jetson-210_Linux_R32.7.5_aarch64.tbz2`	Driver package — bootloader, kernel, and the flashing tools themselves

File	What it is
`Tegra_Linux_Sample-Root-Filesystem_R32.7.5_aarch64.tbz2`	The Ubuntu 18.04 filesystem that will run on the board
`overlay_32.7.5_PCN211181.tbz2`	Support for the newer module revision. **Also fixes DisplayPort configuration and enables the SD-card slot by default**
`tegra210-p3448-0002-p3449-0000-b00.dtb`	The device tree. `p3448-0002` is exactly your module — the 4 GB eMMC production variant

> **The overlay and the .dtb are the steps most people miss**
> Skipping them gives you a board that flashes "successfully" and then has DisplayPort problems and a non-functioning SD-card slot. They are two extra commands in Part 4 — do not omit them.

Part 3 · Prepare the host NVIDIA

```
sudo apt update
sudo apt install -y qemu-user-static abootimg nfs-kernel-server \
                    libxml2-utils binutils python3 device-tree-compiler \
                    lbzip2 cpio udev dosfstools usbutils
```

Put all four downloaded files in one working directory, for example `~/nvidia/`.

> **Use a native Linux filesystem**
> Extract into your Linux home directory, never onto an NTFS or exFAT mount. The root filesystem carries Linux ownership, permission bits, symlinks and device nodes — none of which survive on a Windows filesystem. Extract in the wrong place and the flash either fails or produces a broken system.

Part 4 · Build the flash package TANNA

Run these in order, from the directory containing the four files.

```
tar xf Jetson-210_Linux_R32.7.5_aarch64.tbz2

cd Linux_for_Tegra/rootfs
sudo tar xpf ../../Tegra_Linux_Sample-Root-Filesystem_R32.7.5_aarch64.tbz2

cd ../
sudo ./apply_binaries.sh

cd ../
tar xpf overlay_32.7.5_PCN211181.tbz2
cp tegra210-p3448-0002-p3449-0000-b00.dtb Linux_for_Tegra/kernel/dtb/

cd Linux_for_Tegra/
```

Command	What it does
`tar xf Jetson-210...`	Creates the `Linux_for_Tegra/` directory
`sudo tar xpf ...Sample-Root-Filesystem...`	Unpacks Ubuntu into `rootfs/`. The `p` flag preserves permissions — essential
`sudo ./apply_binaries.sh`	Merges NVIDIA's drivers into the rootfs. Takes several minutes. **Must finish with no errors**
`tar xpf overlay_...`	Applies the module-revision overlay
`cp ...dtb kernel/dtb/`	Installs the device tree for this carrier

Optional — pre-create your user so the board skips the first-boot setup wizard:

```
sudo ./tools/l4t_create_default_user.sh -u echopath -p <password> -a --accept-license
```

Part 5 · Recovery mode TANNA

5.1 Enter force recovery mode

1. **Disconnect power** from the Eagle 101
2. Use a jumper to connect the **FC_REC pin (3)** to the **GND pin (4)**
3. Connect the **USB Type-B** end of the cable to the board, and the **Type-A** end to the host PC
4. Apply power
5. After about five seconds, remove the jumper

Tanna's page has a photograph of the pin header — check it for the physical location of pins 3 and 4 before you start.

5.2 Verify before going further

```
lsusb
```

You must see:

```
ID 0955:7f21 NVidia Corp.
```

> **Do not run the flash command until you see that line**
> If it is absent, in order of likelihood: the USB cable is charge-only, the jumper was on the wrong pins, or power was applied before the jumper was fitted. Try a different cable first.

Part 6 · Flash TANNA

```
sudo ./flash.sh jetson-nano-devkit-emmc mmcblk0p1
```

Ten to twenty minutes. A great deal of text scrolls past — that is normal. Success looks like:

```
*** The target ... has been flashed successfully. ***
```

> **Do not interrupt power or USB while this runs**
> This is the only genuinely delicate moment in the entire process.

Argument	Meaning
`jetson-nano-devkit-emmc`	The board configuration. Selects the device tree and pin configuration for this carrier
`mmcblk0p1`	Root filesystem goes on the module's **16 GB eMMC** — see Part 9 for the space consequences

---

Part 7 · First boot

1. Power off, unplug the USB cable
2. Connect HDMI, keyboard and power
3. Complete the Ubuntu setup wizard if it appears

```
cat /etc/nv_tegra_release        # expect R32, revision 7.5
ls /dev/video0                   # camera present?
lsblk                            # is the SD card visible? (expect mmcblk1)
```

The overlay from Part 2 should make the SD-card slot work with no further configuration.

Part 8 · Install the SDK components ECHOPATH

Flashing installs the operating system only. CUDA, cuDNN, TensorRT and OpenCV come afterwards over the network.

> **Do not run** `sudo apt install nvidia-jetpack`
> The full meta-package is roughly 14 GB installed and **will not fit** on a 16 GB eMMC alongside the operating system. It pulls samples, documentation, VisionWorks, VPI, DeepStream and Nsight — none of which EchoPath uses.

INSTALL SELECTIVELY — ABOUT 6 GB

```
sudo apt update
sudo apt install -y \
    cuda-toolkit-10-2 \
    libcudnn8 libcudnn8-dev \
    tensorrt python3-libnvinfer python3-libnvinfer-dev \
    nvidia-opencv
```

Package	Why EchoPath needs it
`tensorrt`, `python3-libnvinfer`	`trtexec` builds the engines; the Python bindings run them. The core of the deployment
`cuda-toolkit-10-2`	TensorRT depends on it, and `pycuda` will not compile without the headers
`libcudnn8`	TensorRT dependency
`nvidia-opencv`	OpenCV 4.1.1 **built with GStreamer** — the only build that can open the CSI camera

Part 9 · Move the bulk onto the SD card ECHOPATH

Rootfs is on the 16 GB eMMC, so space is tight from the start. Tanna's guide does not document SD-card rootfs for this carrier, so the practical approach is to relocate the large directories.

```
lsblk                                  # identify the card, normally mmcblk1

sudo mkfs.ext4 /dev/mmcblk1p1          # ONLY if the card is empty - this erases it
sudo mkdir -p /mnt/sd
sudo mount /dev/mmcblk1p1 /mnt/sd
echo '/dev/mmcblk1p1  /mnt/sd  ext4  defaults,noatime  0  2' | sudo tee -a /etc/fstab

# Swap on the CARD, never on the eMMC. Required before any TensorRT engine build.
sudo fallocate -l 8G /mnt/sd/swapfile
sudo chmod 600 /mnt/sd/swapfile
sudo mkswap /mnt/sd/swapfile
sudo swapon /mnt/sd/swapfile
echo '/mnt/sd/swapfile  none  swap  sw  0  0' | sudo tee -a /etc/fstab

# Headless - frees roughly 1 GB of the 4 GB total
sudo systemctl set-default multi-user.target

# Full performance
sudo nvpmodel -m 0
sudo jetson_clocks

# Audio for the spoken alerts (the board has no analog codec - USB dongle required)
sudo apt install -y espeak libespeak1 alsa-utils

sudo reboot
```

`noatime` matters: without it, every file *read* also writes to the card, shortening its life considerably.

Keep the EchoPath project, `onnx/` and `engines/` under `/mnt/sd/`.

THEN THE PYTHON ENVIRONMENT

```
python3 -m venv --system-site-packages ~/echopath-env
source ~/echopath-env/bin/activate
export PATH=/usr/local/cuda/bin:$PATH
export CUDA_ROOT=/usr/local/cuda
pip install pycuda==2021.1 pyttsx3==2.90
```

`--system-site-packages` is essential — it lets the virtual environment see JetPack's `tensorrt` and `cv2` while still giving you somewhere to install `pycuda`.

Part 10 · Verify everything before trusting it

- ☐ `cat /etc/nv_tegra_release` → R32, revision 7.5
- ☐ `free -h && df -h /` → 8 GB swap active, sensible free space
- ☐ `python3 -c "import tensorrt; print(tensorrt.__version__)"` → 8.2.x
- ☐ `python3 -c "import cv2; print(cv2.__version__)"` → 4.1.1
- ☐ `python3 -c "import cv2; print('GStreamer' in cv2.getBuildInformation())"` → **True**
- ☐ `python3 -c "import pycuda.autoinit; print('ok')"`
- ☐ `nvcc --version` → CUDA 10.2
- ☐ Camera: `gst-launch-1.0 nvarguscamerasrc num-buffers=100 ! nvvidconv ! fakesink -v`
- ☐ Audio: `aplay -l` lists the USB dongle, and a test WAV is audible
- ☐ Fan spins after `jetson_clocks`

> **The GStreamer check is the one that matters most**
> If it returns False, the CSI camera will never open. The usual cause is someone having run `pip install opencv-python`, which shadows JetPack's build. Never install OpenCV via pip on this device.

BACK UP THE WORKING CARD IMMEDIATELY

```
# from the Linux host, SD card in a reader
sudo dd if=/dev/sdX of=/media/pendrive/echopath_working.img bs=4M status=progress
```

A corrupted card then becomes a twenty-minute restore rather than repeating this entire guide.

Troubleshooting

Symptom	Cause and fix
`lsusb` shows no `0955:7f21`	Charge-only USB cable (most common), wrong pins jumpered, or power applied before the jumper. Try a different cable first
`apply_binaries.sh` errors	Extracted onto a non-Linux filesystem, or the `p` flag was omitted from `tar xpf`. Start again in a Linux directory
Flash stops partway	Bad cable, a USB hub, or a virtual machine. Use a port directly on the host and repeat Part 5
Flashes, but no camera	The overlay or the `.dtb` copy was skipped in Part 4
SD card slot not working	Same cause — the overlay is what enables it

Symptom	Cause and fix
"No space left on device" during apt	You ran `nvidia-jetpack` instead of the selective list in Part 8
`trtexec` dies with no message	Out of memory. Check `dmesg | grep -i "killed process"`. Needs headless boot, 8 GB swap, `--workspace=512`, one engine at a time
Random reboots	Under-powered supply. Needs a genuine 5 V / 4 A

> **A failed flash does not damage the board**
> Recovery mode is re-enterable. Repeat Part 5 and Part 6 as many times as needed.

After this: building the EchoPath engines

Copy `onnx/*.onnx` from the workstation, then build on the board — a `.plan` file is not portable across TensorRT versions or GPU architectures.

```
/usr/src/tensorrt/bin/trtexec --onnx=onnx/yolov8n_640.onnx --fp16 --verbose \
    --workspace=512 --saveEngine=engines/yolo_640_fp16.plan

/usr/src/tensorrt/bin/trtexec --onnx=onnx/midas_small_256.onnx --fp16 \
    --workspace=512 --saveEngine=engines/midas_256_fp16.plan
```

FP16, never INT8. The Nano's Maxwell GPU is compute capability 5.3 and lacks the DP4A instruction. `--int8` warns and silently falls back, producing an engine that looks quantised and performs like FP16. Build one at a time — concurrent builds exhaust the 4 GB.

EchoPath — definitive Eagle 101 flashing guide · prepared 14 August 2026 · unit serial 1010425035310.
Board configuration, recovery pin numbers, file list and command sequence are taken from Tanna TechBiz's official flashing guide. SDK selection, SD-card relocation and engine-build steps are specific to this project. Supersedes *EchoPath_Jetson_BringUp_Runbook.pdf* and *Jetson_Setup_Simple_Guide.pdf*, both of which contained placeholders for the values now confirmed here.